UNIT 13 75 min

Files, CSV, and JSON

Read and write text files safely, then load and save the two formats that real data uses: CSV and JSON.

By the end of this unit you will be able to

- Open, read, and write text files with `with open(...)`
- Choose the right file mode: `r`, `w`, `a`
- Read a CSV file row by row with the `csv` module
- Convert between Python data and JSON with `json`
- Explain why file paths and encodings can break a program, and how to avoid it

Why files

Variables disappear when a program ends. Files keep data between runs. Almost every real data task starts with reading a file, and most results end up written to one.

Opening files with with

`open(path, mode)` returns a file object. Use it inside a `with` block. The file is closed automatically when the block ends, even if an error happens inside it.

Mode	Meaning	If the file is missing
`"r"`	Read text	Error: `FileNotFoundError`
`"w"`	Write text; replaces the old contents	Creates the file
`"a"`	Append to the end	Creates the file

write_read.py

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("First line\n")
    f.write("Second line\n")

with open("notes.txt", "r", encoding="utf-8") as f:
    text = f.read()
print(text)
```

> **Note**
> In the browser sandbox, files live in a temporary in-memory folder. They last until the page is reloaded. On your computer they are saved to disk.

Reading line by line

A file object can be looped over. Python reads one line at a time, so this works even for files too large to fit in memory. Each line keeps its newline character; `.strip()` removes it.

lines.py

```
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("Third line\n")

with open("notes.txt", encoding="utf-8") as f:
    for number, line in enumerate(f, start=1):
        print(number, line.strip())
```

Paths

A path that does not start with a slash is relative to the folder the program runs from. Print `os.getcwd()` if you are unsure. `pathlib` (Unit 15) gives a cleaner way to build paths that work on Windows, macOS, and Linux.

Watch out

Windows paths contain backslashes. Use a raw string (`r"C:\data\grades.csv"`) or forward slashes (`"C:/data/grades.csv"`).

CSV: rows of comma-separated values

CSV is the most common export format for spreadsheets and databases. The `csv` module handles quoting and commas inside values for you. Always pass `newline=""` when opening a CSV file for writing.

csv_demo.py

```
import csv
import io

# A small CSV held in memory so this runs anywhere.
data = io.StringIO("name,score\nAna,91\nBen,84\n")

reader = csv.DictReader(data)
for row in reader:
    print(row["name"], int(row["score"]) + 1)
```

With `DictReader`, each row is a dictionary keyed by the header. Values always arrive as text, so convert numbers with `int()` or `float()`.

csv_write.py

```
import csv

rows = [{"item": "pencil", "qty": 4}, {"item": "eraser", "qty": 2}]
with open("stock.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["item", "qty"])
    writer.writeheader()
    writer.writerows(rows)

with open("stock.csv", encoding="utf-8") as f:
    print(f.read())
```

JSON: nested data

JSON stores dictionaries, lists, strings, numbers, booleans, and null. Web APIs and configuration files use it. `json.dumps()` turns Python data into a JSON string, and `json.loads()` turns a string back into Python. The `json.dump()` and `json.load()` versions read and write files directly.

json_demo.py

```
import json

student = {"name": "Maya", "grades": [91, 88], "active": True, "advisor": None}

text = json.dumps(student, indent=2)
print(text)

back = json.loads(text)
print(back["grades"][1])
```

Python	JSON
`dict`	object `{ }`
`list` / `tuple`	array `[ ]`
`str`	string
`int` / `float`	number
`True` / `False`	`true` / `false`
`None`	`null`

> **Tip**
> Notice that JSON uses lowercase `true` and `null`. Converting back to Python turns them into `True` and `None`.

Handling file errors

Files go missing and paths change. Combine what you learned in Unit 11 with `open()`.

safe_read.py

```
def read_or_default(path, default=""):
    try:
        with open(path, encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        return default

print(repr(read_or_default("does_not_exist.txt", "no file")))
```

Predict the output THINK FIRST

Think about what each file operation leaves behind.

```
with open("a.txt", "w") as f:
    f.write("hi")
with open("a.txt") as f:
    print(f.read())
```

Answer: `hi`

```
with open("a.txt", "a") as f:
    f.write("!")
with open("a.txt") as f:
    print(f.read())
```

Answer: `hi!`
Append adds to the end. It does not replace the file.

```
import json
print(json.dumps({"ok": True, "x": None}))
```

Answer: `{"ok": true, "x": null}`

```
import json
print(json.loads("[1, 2, 3]")[2])
```

Answer: `3`

Practice

▼ **Exercise 1 — Count lines**

Write a program that writes three lines of text to `log.txt`, then reopens the file and prints how many lines it has.

▼ Exercise 2 — Average from CSV

Using the in-memory CSV below, print the average score as a float.

```
name,score
Ana,91
Ben,84
Cy,77
```

▼ Hint

Collect the scores into a list inside the loop, converting each one with `float()`.

Discussion

1. Why does `with open(...)` matter? What could go wrong with a plain `open` and no `close`?
2. A CSV export opens correctly in a spreadsheet but breaks in your code. What are three things you would check first?
3. JSON and CSV both store data as text. What does each handle well, and what does each handle badly?

Key terms

File object

The object returned by `open()` that provides read and write methods for a file.

with statement

A block that acquires a resource and guarantees it is released when the block ends.

Mode

The letter(s) passed to `open()` that set reading, writing, or appending, such as `"r"`, `"w"`, or `"a"`.

Encoding

The rule for turning characters into bytes, such as UTF-8. Mismatched encodings garble text.

Relative path

A file path measured from the folder the program runs in, rather than from the root of the disk.

CSV

Comma-separated values: plain text with one record per line and commas between fields.

JSON

JavaScript Object Notation: a text format for nested data made of objects, arrays, strings, numbers, booleans, and null.

Serialize

Convert data in memory into a format that can be stored or sent, such as a JSON string.

See all terms in the glossary →